Day 19 - Text vs Real Dates, and Python Functions

Saturday, 7 November 2026 | Swagat (Smooth) | Excel dates + Python functions (return, None, defaults, keyword arguments)

1. Key ideas

- **Real date = number.** Excel stores 3 Nov 2026 as 46329 (11 Mar 2026 is 46092). Format is only the display. Text dates sort alphabetically; real dates sort by value. Check: =ISTEXT(A2) is TRUE for text; =ISNUMBER(A2) is TRUE for a real date. Visual: real dates are right-aligned, text left-aligned; set the cell to General and a real date turns into a number. ISDATE does not exist in Excel (#NAME?).
- **DATEVALUE(A2)** converts text to a date serial number (apply a date format afterwards). Risk: 03-11-2026 can be 3 Nov or 11 Mar, decided by the regional setting. Day 12 or less fails silently; day 13 or more can raise #VALUE!. Safer: build with DATE(year, month, day), or Text to Columns with DMY/MDY. Sanity check: min and max dates.
- **Function parts:** name, parameter (slot in def), argument (value passed in the call), work (body), return value. **return** is a keyword, not a function. It hands a value back to the *caller* (the code that called it), not to the user. **print** only shows text on screen. No return statement gives back None.
- **None** is a real object (NoneType). None + 10 raises TypeError. Code after return never runs (unreachable). return outside a function raises SyntaxError. A SyntaxError stops the whole file before any line runs.
- **Empty list:** sum([]) is 0 and len([]) is 0, so sum/len raises ZeroDivisionError. Returning 0 is dishonest (same as a real average of [0, 0]). Return None, and the caller checks *is None* before using it. Loud failure beats a silent wrong number.
- **Defaults and keywords:** a default is written in def (exponent=2) and used only when the caller gives nothing. A keyword argument is written in the call (exponent=3) and matched by *name*, so order does not matter. Positional first, then keyword. Parameters with defaults come last in def.

2. Examples (Python run and checked; Excel formulas not run)

```
def get_total(numbers):
    total = 0
    for n in numbers:
        total = total + n
    return total

print(get_total([2, 4, 6]))   # 12
```

```
def average(numbers):
    if len(numbers) == 0:
        return None
    return sum(numbers) / len(numbers)

print(average([2, 4, 6]))   # 4.0
print(average([]))          # None
```

```
def power(base, exponent=2):
    return base ** exponent

power(4)                    # 16, default used
power(2, 3)                 # 8, positional
power(exponent=3, base=2)   # 8, keyword, order does not matter
power(2, exponent=5)        # 32, mixed
power(base=2, 3)            # SyntaxError: positional argument follows keyword argument
power(2, base=3)            # TypeError: got multiple values for argument 'base'
```

3. My mistakes and the fix

Mistake	Fix
Wrote ISDATE(A2) in Excel.	ISDATE does not exist (#NAME?). Use ISNUMBER(A2) for a real date and ISTEXT(A2) for text.
Said DATEVALUE problem was the mm-dd-yyyy format.	Format is only display. The cell must hold a number. The risk is day/month order when converting text.
Said return gives output to the user, and print calls the function.	return hands the value to the caller. print only shows it. The call is name(...) with brackets.
Pointed to the print line as the output line.	The output line is return total. print sits outside the function.
Said 'it will return None' when asked what it prints, and 'result has no value'.	It prints None. result holds None, which is a value (NoneType).
Left print(total) after return; called return a function.	Dead code never runs. return is a keyword, not a function.
Chose return 0 for an empty list, 'I think 0 is fine'.	0 looks like a real average. Return None and make the caller check is None.
Pasted print(average([]) with a missing bracket.	SyntaxError: '(' was never closed; nothing runs. Paste what Colab really shows.
Called a keyword call 'default argument'; said order follows the return line.	Defaults live in def; keywords live in the call and match by name, not order.
Labelled power(3) 'mixed' and power(2, exponent=5) 'default using'.	power(3) is positional plus default. power(2, exponent=5) is mixed (positional plus keyword).

Mistake	Fix
Gave TypeError for power(base=2, 3) twice; wrote 4 squared, not 16.	Read left to right: positional after keyword is SyntaxError. Always give the value (4 ** 2 = 16).

4. How I improved

- Recall of return versus print was correct at the second attempt: hands a value back to the caller, print only shows, no return gives None, return outside a function is a SyntaxError.
- Reasoned unprompted why None beats 0 for an empty average, with the [0, 0] example and the absent-versus-scored-zero case.
- Started reading call patterns left to right and gave the exact SyntaxError text for argument order.

5. Practice to retry (no solutions)

- Write max_of(numbers) that returns the largest value and None for an empty list. Test it with the order reversed.
- Predict output or error name: def f(x, y=3): return x * y with calls f(2), f(2, 4), f(y=5, x=2), f(y=5, 2).
- A cell shows 12-05-2026, left-aligned. Give two checks that it is text, the conversion formula, and what could go wrong.
- Fix: def show(n): print(n * 2) so that show(4) + 1 gives 9. Explain the change in one sentence.

6. Recall questions

- Why do text months sort wrongly, and which two formulas tell text from a real date?
- Why can 03-11-2026 convert silently to the wrong date, and what is the safer build?
- Parameter versus argument: define each with one example.
- return versus print versus no return: what does each give the caller?
- Default argument versus keyword argument: where is each written, and what error does a positional after a keyword raise?

7. Open items

- Finish Functions: is_prime and the grade counts function; practice solving alone.
- Retest the profit-case answer lines 4 and 5 (three causes marked as hypotheses; data to pull, action, target).
- Later in the plan: Exceptions and modules, Files and CSV, README/.gitignore, Git recall and conflict redo, XLOOKUP/INDEX+MATCH recall.